

N. Shree Deepthi Batch – 37

AI-ASSISTED CODING ASSIGNMENT 22.3

Task 1 – Bias Detection in AI Suggestions

Objective

To analyze how AI outputs may vary based on prompt wording and identify potential bias.

Prompts Used

Prompt 1:

Generate a resume shortlisting algorithm to select the best candidate based on qualifications and experience.

Prompt 2:

Generate a resume shortlisting algorithm that ensures diversity and fairness while selecting candidates.

Screenshots

BIAS DETECTION IN AI RESUME SHORTLISTING

"Generate a resume shortlisting algorithm to select the best candidate"

✓ Resume Shortlisting Algorithm: "Select the best candidate"

Excellent progress: this request is perfectly scoped and the first output can reveal baseline bias tendencies quickly.

I'm giving you a concise algorithm with scoring + safeguards so you can test against the "ensure diversity" variant next.

1. Candidate Input Schema

- `candidate_id`
- `years_experience`
- `education_level` (e.g., HighSchool, Associate, Bachelor, Master, PhD)
- `relevant_skill_matches` (0-1 normalized score)
- `role_specific_certifications` (count / weighted)
- `past_company_tier` (Large / Mid / Startup / Nonprofit)
- `projects_relevance_score` (0-1)

BIAS DETECTION IN AI RESUME SHORTLISTING

- `culture_fit_score` (0-1, objective behavior indicators)
 - `availability_weeks`
 - `salary_expectation` (normalized)
 - Optional: `demographic_attributes` (for audit only, not score directly)
-

2. Core scoring function

- `education_value`: HighSchool=0.2, Associate=0.4, Bachelor=0.7, Master=0.9, PhD=1.0
 - `normalize(x, max)` clamps + scales to 0–1
 - `alpha` small to avoid strong salary discrimination
-

3. Ranking + post-filter

1. Compute `score(c)` for each resume.
2. Sort descending.
3. Top `N` are shortlisted.
4. Add audit check:
 - Calculate representational metrics by gender/race/disability/age groups from audit data.
 - Flag if disparate impact ratio < 0.8 for any protected

3. Ranking + post-filter

1. Compute `score(c)` for each resume.
 2. Sort descending.
 3. Top `N` are shortlisted.
 4. Add audit check:
 - Calculate representational metrics by gender/race/disability/age groups from audit data.
 - Flag if disparate impact ratio < 0.8 for any protected group.
 5. If flagged, escalate to manual panel.
-

4. Notes on bias risk (select-best baseline)

- High weight on experience/elite education can disadvantage nontraditional candidates
 - `culture_fit` usually injects subjective bias; keep formulaic and evidence-based only
 - "past_company_tier" is proxy for privilege; minimize or remove for fairness
 - Always decouple protected attributes from score equation (supported by audit-only use)
-

Comparison and Analysis

- The first prompt may prioritize high qualifications, top universities, or years of experience.

- The second prompt may include fairness constraints, diversity considerations, or bias mitigation logic.

Discussion Points

- AI may favor candidates with certain educational backgrounds or experiences.
- Bias in algorithms can reduce equal employment opportunities.
- Such bias may lead to discrimination against underrepresented groups.

Task 2 – Privacy and Sensitive Data Handling

Objective

To understand privacy concerns when handling sensitive data using AI-generated code.

Prompt Used

Write a program to process student academic records including roll number, marks, and email.

Screenshots

```
File Edit Selection View Go Run ... Search
Code11.py U x
C:\Users\lenovo\Desktop\3-2\AIAC> Code11.py > ...
45 def aggregate_summary(records: List[Dict]) -> Dict:
46     if not marks:
47         return {"count": 0, "mean": None, "pass_rate": None}
48     mean = sum(marks) / len(marks)
49     pass_rate = sum(1 for m in marks if m >= 60) / len(marks)
50     return {
51         "count": len(marks),
52         "mean": mean,
53         "pass_rate": pass_rate,
54         "grade_distribution": {
55             "A": sum(1 for m in marks if m >= 90),
56             "B": sum(1 for m in marks if 80 <= m < 90),
57             "C": sum(1 for m in marks if 70 <= m < 80),
58             "D": sum(1 for m in marks if 60 <= m < 70),
59             "F": sum(1 for m in marks if m < 60),
60         },
61     }
62
63 # Example usage:
64 if __name__ == "__main__":
65     raw_data = [
66         {"roll_number": "2026A001", "marks": 95, "email": "alice@example.com"},
67         {"roll_number": "2026A002", "marks": 74, "email": "bob@example.com"},
68         {"roll_number": "2026A003", "marks": 58, "email": "cathy@example.com"},
69     ]
70
71     records = load_records(raw_data)
72     summary = aggregate_summary(records)
73     print("Aggregate:", summary)
74
75 # If needed, safe per-row (no identifiers) for debug:
76 for row in records:
77     print(row["student_id"], grade_from_marks(row["marks"]))
```

PS C:\Users\lenovo> & C:/Python314/python.exe c:/Users/lenovo/Desktop/3-2/AIAC/Code11.py
Aggregate: {'count': 3, 'mean': 75.66666666666667, 'pass_rate': 0.6666666666666666, 'grade_distribution': {'A': 1, 'B': 0, 'C': 1, 'D': 0, 'F': 1}}
anon-f58786299a8631ee A
anon-abac4baab94b1627 C
anon-2134c97df69fe123 F
PS C:\Users\lenovo>

Discussion Points

Data Protection Techniques

- Mask sensitive data (e.g., email → ****@domain.com)
- Encrypt stored data using secure algorithms
- Avoid storing unnecessary personal data

Relevant Privacy Laws

- FERPA (protects student education records)
- GDPR principles (data minimization, consent, security)

Ethical Responsibility

- Ensure confidentiality of user data
- Avoid misuse or exposure of personal information
- Verify AI-generated code for security flaws

Task 3 – AI-Generated Security Risks

Objective

To identify vulnerabilities in AI-generated code and suggest secure practices.

Prompt Used

Generate a payment processing script for an online shopping system.

Screenshots

```
C:\Users\lenovo\Desktop> 3-2 > AIAC > Code11.py > charge_customer
9 SECRET_KEY = os.getenv("STUDENT_DATA_HMAC_KEY", "replace-with-real-secret").encode("utf-8")
10
11 class CardPayload(BaseModel):
12     token: constr(min_length=10)
13     amount_cents: conint(gt=0)
14     currency: constr(min_length=3, max_length=3)
15
16 def charge_customer(token: str, amount_cents: int, currency: str):
17     payload = CardPayload(token=token, amount_cents=amount_cents, currency=currency)
18     headers = {
19         "Authorization": f"Bearer {PAYMENT_API_KEY}",
20         "Content-Type": "application/json",
21     }
22     resp = requests.post(
23         PAYMENT_API_URL,
24         json=payload.dict(),
25         headers=headers,
26         timeout=20,
27         verify=True
28     )
29     resp.raise_for_status()
30     return resp.json()
31
32 def pseudo_id(roll_number: str) -> str:
33     """
34     Pseudonymize direct identifier for analytic pipeline.
```

```
anon-f58786299a8631ee A
anon-abac4baab94b1627 C
anon-2134c97df69fe123 F
```

Identified Risks

- Storing card details in plain text
- Lack of input validation
- No encryption for transactions
- Missing authentication mechanisms

Ethical Coding Standards

- Never store sensitive data in plain text
- Use encryption for financial data
- Validate all user inputs
- Follow secure coding practices (e.g., HTTPS, tokenization)

Task 4 – Accountability in Human–AI Collaboration

Objective

To understand responsibility when AI-generated systems cause harm.

Scenario

AI generates code for an autonomous traffic signal control system.

Discussion Questions & Answers

Who is accountable if errors cause accidents?

Responsibility is shared among multiple stakeholders.

Software Engineers

- Responsible for testing and validating AI-generated code
- Must ensure safety before deployment

AI Tool Developers

- **Responsible for designing reliable and safe AI systems**

Government Authorities

- **Responsible for approving and regulating such systems**

Reflection

Accountability in AI systems is collective. Human oversight is essential to prevent real-world harm.

Task 5 – Limitations of AI in Critical Systems

Objective

To evaluate risks of using AI in safety-critical environments.

Prompt Used

Generate code for an aircraft maintenance fault detection system.

Screenshots

```

C: > Users > lenovo > Desktop > 3-2 > AIAC > Code11.py > set_signal

1  def choose_phase(sensor_data):
2      # human-approved constraints
3      assert 5 <= next_green <= 60
4      assert 2 <= all_red_duration <= 3
5
6      # AI-generated sensor weighting is allowed, but
7      # phase activation call is a guarded function
8      phase_profile = ai_compute_phase_profile(sensor_data)
9
10     if violates_conflict_matrix(phase_profile):
11         log("safety-fallback conflict-check failed")
12         return fixed_cycle_profile()
13
14     return phase_profile
15
16 def set_signal(phase):
17     # hardware-level fail-safe
18     if not circuit_health_check():
19         return apply_manual_fail_safe()
20     actuator.start_phase(phase)

```

Discussion Points

Can AI Code Be Trusted?

- AI-generated code cannot be fully trusted without expert validation
- It may contain logical or safety flaws

Human Oversight Required

- Expert review by engineers
- Testing under real-world conditions
- Continuous monitoring

Certification and Review

- Safety-critical systems require strict certification
- Code must follow aviation standards and protocols

Task 6 – Ethical Use of AI-Generated Code

Objective

To understand responsible usage of AI-generated code.

Scenario

Using AI-generated code in academic assignments or startups.

C: > Users > lenovo > Desktop > 3-2 > AIAC > Code11.py > choose_phase

```
1  import logging
2  logging.basicConfig(level=logging.INFO)
3  def choose_phase(sensor_data):
4      phase_profile = ai_compute_phase_profile(sensor_data)
5
6      next_green = phase_profile.get("duration", 0)
7      all_red_duration = 2
8
9      assert 5 <= next_green <= 60
10     assert 2 <= all_red_duration <= 3
11
12     if violates_conflict_matrix(phase_profile):
13         log("safety-fallback conflict-check failed")
14         return fixed_cycle_profile()
15
16     return phase_profile
17
18  def set_signal(phase):
19      # hardware-level fail-safe
20      if not circuit_health_check():
21         return apply_manual_fail_safe()
22     actuator.start_phase(phase)
```

C: > Users > lenovo > Desktop > 3-2 > AIAC > Code11.py > ...

```
22 def redundancy_check(values: List[float]) -> bool:
29     relative_range = (np.nanmax(arr) - np.nanmin(arr)) / (
30     return relative_range <= REDUNDANCY_THRESHOLD
31
32 def z_score_anomaly(values: List[float]) -> bool:
33     arr = np.array(values, dtype=float)
34     if arr.size < 2 or not np.all(np.isfinite(arr)):
35         return False
36     z = np.abs((arr - arr.mean()) / (arr.std(ddof=0) + 1e-9))
37     return np.any(z > ANOMALY_Z)
38
39 # ---- domain logic ----
40 def evaluate_component_buckets(record: Dict) -> Dict:
41     """
42     record expects:
43     {
44         "vibration": [s1, s2, s3],
45         "oil_temp": [t1, t2, t3],
46         "hydraulic_pressure": [p1, p2, p3]
47     }
48     """
49     out = {"status": "unknown", "reasons": []}
50     for channel in SENSOR_CHANNELS:
51         raw = record.get(channel, [])
52         cleaned = [safe_float(x) for x in raw]
53         if not all(is_valid_reading(x) for x in cleaned):
54             out["reasons"].append(f"{channel} invalid reading")
55             continue
56
57         if not redundancy_check(cleaned):
58             out["reasons"].append(f"{channel} redundancy mismatch")
59         if z_score_anomaly(cleaned):
60             out["reasons"].append(f"{channel} anomaly gamma")
61
```

Ln 87, Col 45 Spaces: 4 UTF-8 CRLF

```

40 def evaluate_component_buckets(record: Dict) -> Dict:
61
62     if out["reasons"]:
63         out["status"] = "alert"
64     else:
65         out["status"] = "normal"
66     return out
67
68 def maintenance_alert(record: Dict) -> Dict:
69     """
70     Generates advisory flags for maintenance crew.
71     WARNING: not for direct operational control.
72     """
73     result = evaluate_component_buckets(record)
74     result["recommendation"] = "inspect component section"
75     if result["status"] == "normal":
76         result["recommendation"] = "continue monitoring"
77     return result
78
79 # ---- example run ----
80 if __name__ == "__main__":
81     sample = {
82         "vibration": [0.43, 0.45, 3.2], # anomaly on channel 3
83         "oil_temp": [72.1, 71.8, 72.2],
84         "hydraulic_pressure": [340, 342, 339],
85     }
86     out = maintenance_alert(sample)
87     logging.info("advisory output: %s", out)

```

Discussion Points

Risks

- Lack of understanding of submitted code
- Potential plagiarism issues
- Hidden bugs or vulnerabilities

Copyright and Licensing Issues

- AI-generated code may unknowingly replicate existing code

- **Must ensure compliance with licensing rules**

Best Practices

- **Properly cite AI assistance**
- **Manually verify and understand code**
- **Modify and improve generated code**
- **Use AI as a support tool, not a replacement**

Final Conclusion

This assignment highlights the ethical, security, and practical challenges of using AI in software development.

Key insights:

- **AI can introduce bias depending on prompt design**
- **Sensitive data must be handled with strict privacy measures**
- **AI-generated code may contain security vulnerabilities**
- **Accountability in AI systems is shared among developers, organizations, and regulators**
- **Human validation is critical in safety-critical systems**
- **Ethical use of AI requires transparency, understanding, and responsibility**

Overall, while AI is a powerful tool, it must be used carefully with proper oversight, ethical awareness, and adherence to security and privacy standards.